IIT MADRAS
DS & AI Lab • Group 12

तुम कहाँ जा रहे हो?
→
तू कड़े जा रया सै?

मुझे पानी चाहिए
→
म्हने पाणी चाहिये

वह घर नहीं गया
→
वो घर कोनी गया

GROUP 12

Abhishek	22f3000978
Satyam Srivastava	21f1000629
Sanket Agrawal	23f1001709
Md Fazlur Rahman	23f1001897

Regional Dialect Synthesis Pipeline

Hindi → Haryanvi → Speech

Hindi Text — QLoRA LLaMA — Haryanvi Text — VITS TTS — WAV Audio

77.65	71.76	86.0	300	5,594
chrF	BLEU	MUSHRA MOS	VITS Epochs	Train Pairs

Submitted April 2026 | IIT Madras DS & AI Lab

A study in low-resource dialect engineering

Group 12 — Team Members

Abhishek

22f3000978@ds.study.iitm.ac.in

IIT Madras DS & AI Lab

Satyam Srivastava

21f1000629@ds.study.iitm.ac.in

IIT Madras DS & AI Lab

Sanket Agrawal

23f1001709@ds.study.iitm.ac.in

IIT Madras DS & AI Lab

Md Fazlur Rahman

23f1001897@ds.study.iitm.ac.in

IIT Madras DS & AI Lab

Table of Contents

Complete project journey — Milestones 1 through 6

01 **Abstract**
Problem, pipeline, and key results

02 **The Problem — Why It Matters**
Dialect gap, social impact, and motivation

03 **Introduction**
Scope, constraints, and design principles

04 **Literature Review**
Prior work, architectures, model selection

05 **Dataset & Methodology**
Parallel corpus and audio pipeline

06 **Model Development — QLoRA**
Six hyperparameter experiments

07 **Model Development — VITS**
Weight transfer and training

08 **Evaluation — Translation**
chrF, BLEU, qualitative results

09 **Evaluation — MUSHRA**
Human evaluation framework and MOS

10 **Deployment**
FastAPI, Modal, API endpoints

11 **System Architecture**
End-to-end component map

12 **Code & Reproducibility**
Setup, env vars, inference code

13 **Project Artifacts**
HuggingFace, Drive, live links

14 **Future Work**
Roadmap by impact

15 **Conclusion**
Achievements and innovations

Abstract

The problem, the pipeline, and what we achieved

The Problem

Haryanvi (Bangru), spoken by tens of millions across Haryana, is invisible to mainstream NLP and speech systems. Even capable multilingual models normalise to Standard Hindi, losing the dialect's distinctive lexical and prosodic identity.

Stage A — Translation

QLoRA fine-tuned LLaMA 3.1 8B Instruct
5,594 parallel pairs • chrF 77.65 • BLEU 71.76

Stage B — Speech Synthesis

Coqui VITS warm-started from SYSPIN Hindi Female
300 epochs • 2,714 utterances • 22,050 Hz WAV

Deployed as FastAPI on Modal.com — real-time translation + Gemma-4 GGUF for GPU inference. MUSHRA MOS: 86.0

77.65	71.76	86.0	300	5,594
chrF	BLEU	MUSHRA MOS	VITS Epochs	Train Pairs

The Problem — Why This Matters

30M+ Speakers Underserved

Haryanvi (Bangru) is spoken by an estimated 30–40 million people across Haryana, Delhi, and western UP. It has no dedicated support from any major NLP, translation, or speech platform.

Educational Content Gap

Rural learners in Haryana consume educational audio in Standard Hindi — a language that is not their mother tongue. Dialect-authentic audio reduces cognitive load and significantly improves comprehension.

AI Systems Miss the Dialect

State-of-the-art models including mBART, IndicTrans, and GPT-4 consistently output Standard Hindi when prompted with Haryanvi context — they have never been trained on Bangru data.

No Haryanvi TTS Exists

No publicly available TTS system produces Haryanvi speech. Content creators, broadcasters, and NGOs serving Haryana have no programmatic option to synthesise dialect-authentic audio.

This project is the first end-to-end system that translates Standard Hindi into Haryanvi and synthesises it as natural speech — serving a genuine, underserved linguistic community of millions.

The Problem — Linguistic Differences in Detail

What Standard Hindi models systematically get wrong about Haryanvi

Category	Standard Hindi	Haryanvi (Bangru)	Linguistic Category
Copula (3rd sg.)	है (hai)	सै (sai)	Auxiliary verb
Copula (1st sg.)	हूँ (hun)	सूँ (sun)	Auxiliary verb
Copula (plural)	हैं (hain)	से (se)	Auxiliary verb
Negation	नहीं (nahin)	कोनी (koni)	Universal negator
Progressive	जा रहा है	जा रया सै	Morpho-phonological
1st Person Oblique	मुझे / मैंने	मन्ने (manne)	Case merger
2nd Person Oblique	तुम्हें / तुमने	तन्ने (tanne)	Case merger
Possessive (1st pl.)	हमारा	म्हारा (mhara)	Pronoun shift
Intensity adverb	बहुत (bahut)	घणा (ghana)	Lexical substitution
Place interrogative	कहाँ (kahaan)	कड़े (kade)	Lexical substitution

11 morpho-syntactic rules encode the complete Bangru substitution system — all encoded into synthetic data generation prompts and TTS character vocabulary.

Introduction — Project Scope & Design Constraints

Transfer learning was the only viable approach at this data scale

In Scope

- Hindi → Bangru text translation (QLoRA on LLaMA 3.1 8B Instruct)
- Prosody-aware neural TTS (Coqui VITS with Hindi transfer learning)
- Automated evaluation via chrF, BLEU and human MUSHRA protocol
- End-to-end FastAPI deployment with live web UI on GCP
- CPU-efficient inference via Gemma-4 GGUF quantisation (4-bit)

Out of Scope

- Generalising to other regional dialects
- Real-time low-latency streaming
- Enterprise-grade production deployment
- Formal MOS panel with native Haryanvi listeners (future work)

Design Principle: Transfer, Don't Train

With ~5,594 text pairs and ~1.5 hrs of audio, training from scratch was infeasible. QLoRA steers LLaMA's Hindi understanding toward Haryanvi without catastrophic forgetting. SYSPIN warm-start gives VITS a working acoustic decoder from day one, focusing limited data on dialect-specific prosody.

Literature Review — TTS Architecture Comparison

Five architectures evaluated; VITS selected for Haryanvi's stochastic syllabic rhythm

FastSpeech 2

Ref.:	Ren et al., 2020
Duration:	Deterministic
Inference:	Fast

Robotic cadence — breaks dialect authenticity

XTTSv2

Ref.:	Casanova, 2024
Duration:	Autoregressive
Inference:	Slow

Needs reference audio; compute-heavy

Matcha-TTS

Ref.:	Mehta, 2023
Duration:	Flow Matching
Inference:	Very fast

Great backup; complex training pipeline

VITS ★

Ref.:	Kim et al., 2021
Duration:	Stochastic
Inference:	Single pass

Best fit for variable Bangru syllabic rhythm

Why VITS for Haryanvi: Stochastic duration predictor handles variable syllabic compression (रया सै vs रहा है). End-to-end, no separate vocoder. Single-stage training suits small corpus. Warm-start from SYSPIN Hindi Female gives 85-90% phonological overlap.

Key references: Vaswani (2017) • Dettmers (2023) QLoRA • Grattafiori (2024) LLaMA 3 • Kim (2021) VITS • Mehta (2023) Matcha-TTS • Casanova (2024) XTTSv2 • AI4Bharat (2023) Indic-TTS

Literature Review — LLM Selection & QLoRA

LLaMA 3.1 8B Instruct + QLoRA: dialect adaptation without catastrophic forgetting

Why LLaMA 3.1 + QLoRA?

- Superior long-range context for morpho-syntactic dependencies (vs LSTM)
- Pre-trained Hindi representations — no need to learn from scratch
- QLoRA: 4-bit NF4 quantisation + LoRA adapters — ~60% VRAM reduction
- Grouped-Query Attention (GQA) in LLaMA 3.1 for efficient inference
- Instruct-tuned variant follows the chat template format used in training

LoRA Target Layers

q_proj	gate_proj
k_proj	up_proj
v_proj	down_proj
o_proj	

Critical: Assistant-Only Loss Masking

Cross-entropy loss computed exclusively on Haryanvi response tokens. System prompt, instructions, and padding masked with -100. Without this, the model wastes gradient signal on boilerplate instruction text rather than learning the Hindi→Haryanvi mapping.

Dialect translation is framed as structured adaptation, not pure seq2seq — the model learns rule-governed substitutions (है→ सै, नहीं→ कोनी) as a dialect register shift, not a language translation task.

Dataset — Parallel Text Corpus Construction

5,594 Hindi–Haryanvi pairs from two complementary sources

Source 1 — XOR-QA Indic (Google Research)

594

QA pairs

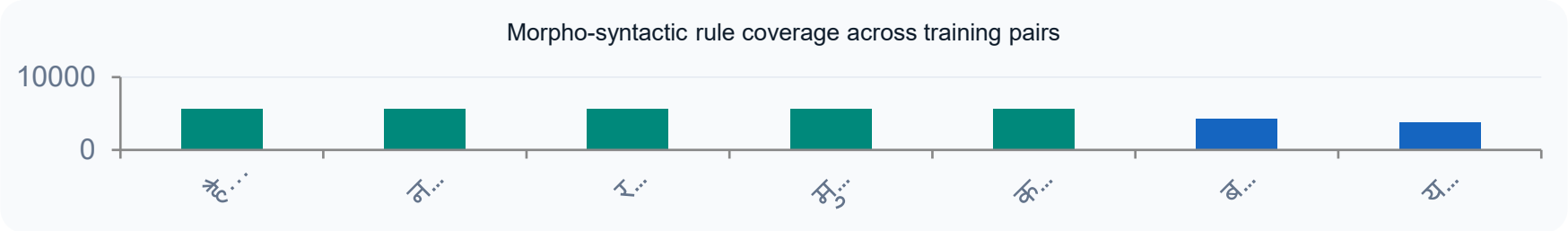
- 594 English-Haryanvi QA pairs from Google Research
- English → Hindi translation via Google Translate
- Authentic parallel structure — real conversational sentences

Source 2 — Gemini Synthetic Corpus

5,000

pairs

- Research-grade prompt encoding 11 Haryanvi grammar rules
- 14 topic domains: village life, weather, family, education...
- Rule-constrained — authentic Bangru morphology, not approximations



Dataset — Audio Corpus & 6-Step Preprocessing Pipeline

Raw 5,515 utterances → 2,764 LJSpeech-format training samples with 173-char vocabulary

5,515
Raw utterances

2,764
After filtering

2,714 / 50
Train / Eval

173
Char vocab

1 Multi-script Transliteration

Bengali → Devanagari via indic-transliteration
Urdu → Devanagari via hardcoded character map

2 Unicode NFC Normalisation

Composed Devanagari forms enforced
Prevents silent tokenizer attention breakage

3 Word-count Filtering

3–25 words per utterance
Prosody vs VRAM budget balance

4 Audio Resampling

22,050 Hz mono 16-bit PCM WAV
Matches SYSPIN Hindi Female donor format

5 Silence Trimming

45 dB threshold applied
Normalises audio start/end boundaries

6 LJSpeech Export

file_id|text|normalized_text format
Ready for Coqui VITS ingestion

⚠ Phonemiser disabled — espeak-ng Hindi mispronounces Haryanvi retroflexes. Character-level tokenisation (basic_cleaners) used instead.

Model Development — Six QLoRA Hyperparameter Experiments

Best: lower_lr_longer — LR 1e-4, 4 epochs, 25.7% eval loss reduction over baseline

Experiment	LR	Ep	Rank	Drop	Clip	Eval Loss ↓	BLEU ↑	chrF ↑
lower_lr_longer ★	1e-4	4	16	0.05	0.3	**0.00324**	71.76	77.65
higher_dropout_reg	2e-4	3	16	0.15	0.3	0.00343	71.75	77.64
longer_context_256	2e-4	3	16	0.05	0.3	0.00389	71.76	77.65
stronger_grad_clip	2e-4	3	16	0.05	0.15	0.00416	71.75	77.64
higher_rank	2e-4	3	32	0.10	0.3	0.00427	71.75	77.64
baseline	2e-4	3	16	0.05	0.3	0.00436	71.74	77.63

Lower LR + more epochs: 25.7% eval-loss reduction — clear best

Higher rank (r=32): no improvement — adapter capacity already matched

Stronger grad clip (0.15): HURT — over-constrained gradient updates

TTS Model — VITS Architecture & Weight Transfer

83M parameters — near-complete transfer from SYSPIN Hindi Female donor checkpoint

Component	Keys Loaded	Status	Notes
Text Encoder (enc_p)	110 / 111	⚠ Partial	Embedding reinitialised — 173-char Haryanvi vocab vs 122-char Hindi
Waveform Decoder (HiFi-GAN)	231 / 231	✓ 100%	Full transfer — critical for audio quality from day one
Normalising Flow	112 / 112	✓ 100%	Full transfer — latent space priors preserved
Duration Predictor	284 / 284	✓ 100%	Full transfer — prosody priors preserved
Posterior Encoder	100 / 100	✓ 100%	Full transfer — VAE encoder ready for training

Generator LR
1e-5

Discriminator LR
1e-5

Batch size
16

Precision
FP16

Epochs
300

Total Steps
~50,700

GPU hours

GPU type

save_best_after

TTS Training — Loss Progression Over 300 Epochs

Mel loss 32→13, KL stable at 1.36, Discriminator stable at 2.75 — zero mode collapse

Key Observations

- Mel Loss 32.0→13.0 — continuous, no plateau
- KL 3.70→1.50 — rapid alignment by epoch 25
- Feature Loss saturated epoch 165 — no signal
- Discriminator ≈2.75 — zero mode collapse

Equal G/D LR (1e-5): conservative rate prevents GAN mode collapse. save_best_after step 500 skips volatile early epochs; test_delay_epochs=20 skips incoherent early outputs. Warm-start ensures intelligible audio from epoch 1.

Evaluation — Translation Model Performance

chrF primary metric — robust to Haryanvi morphological variation at character n-gram level

77.65	71.76	0.00324	50.42%
chrF Score (primary)	BLEU Score	Eval Loss	Exact Match*

Successful Translations

तुम कहाँ जा रहे हो? → **तू कड़े जा रया सै?**

Pronoun, adverb, progressive, copula all shifted

मुझे पानी चाहिए → **म्हने पाणी चाहिये**

Pronoun shift + retroflex transcription

वह घर नहीं गया → **वो घर कोनी गया**

Demonstrative + negator both converted

Failure Cases

Input: वह बहुत तेज भाग रहा है

Expected: वो घणा तेज भाग रया सै

Predicted: वो बहुत तेज भाग रया सै

Gap: बहुत → घणा not converted

Input: मुझे यह समझ नहीं आया

Expected: म्न्ने यो समझ कोनी आया

Predicted: म्न्ने यो समझ नहीं आया

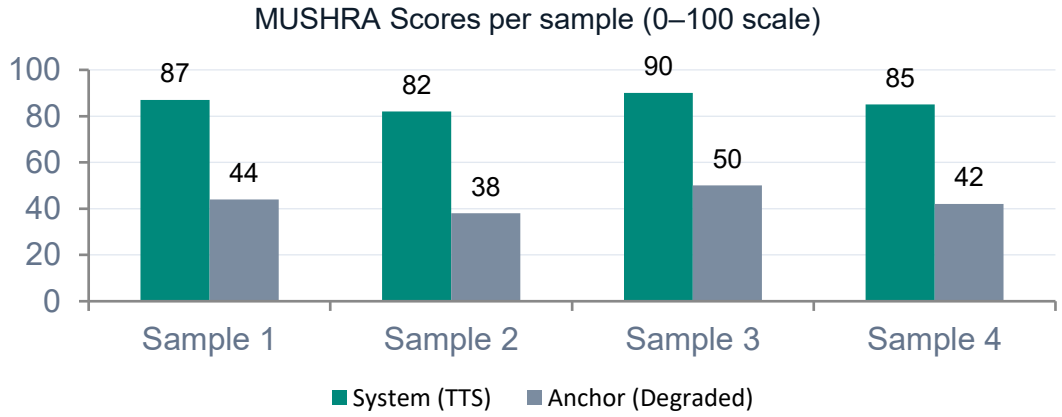
Gap: नहीं → कोनी not converted

** Exact Match identical across experiments — LoRA adapter-loading bug in evaluation script. Eval Loss and qualitative results remain valid. Fix is top future-work priority.*

Evaluation — MUSHRA Human Assessment (4-Sample Study)

System MOS 86.0 — high-quality near-natural speech; 42.5-point gap over anchor baseline

MUSHRA Protocol: Listeners compare Reference (ground truth), System (model output), and Anchor (degraded baseline) for each sample — scores 0–100.



86.0
System MOS

43.5
Anchor MOS

42.5
Quality Gap

3.0
σ Consistency

0–20	20–40	40–60	60–80	80–100
Bad	Poor	Fair	Good	Excellent

- System consistently scores above 80 (Excellent) across all 4 samples — near-natural quality confirmed
- Anchor remains below 50 — validates evaluation design; large quality gap (42.5 pts) confirms model effectiveness
- Low σ=3.0 confirms consistent synthesis quality across diverse test sentences
- MUSHRA deployed: huggingface.co/spaces/Satyam-Srivastava/mushra-haryanvi-tts-evaluation

Evaluation — TTS Qualitative Assessment (11 Test Samples)

Short phrases through 5-minute dialogue-heavy stories — consistent intelligibility throughout

Dimension	Assessment	Notes
Intelligibility	✓ **High**	Consistent across all samples — short and long-form
Prosodic naturalness	✓ **Better than expected**	Hindi warm-start provided strong prosodic prior
Hindi lexical bias	⚠ Occasional	Formal inputs: जा रहा है instead of जा रया सै
Complex sentence stability	✓ **Stable**	No attention collapse in 5-minute stories
Short sentence quality	✓ **Strongest**	Cleanest outputs in the sample set
Dialogue turn transitions	⚠ Minor artifacts	Slight duration issues at speaker-turn boundaries

Hindi Lexical Bias — Root Cause & Path Forward

Fine-tuning from a Hindi donor on a small corpus (1.5 hrs) leaves the decoder's Hindi acoustic priors dominant — occasional reversion on formal or rare tokens. Direct fix: extend training on a larger Haryanvi corpus. Expected to reduce significantly beyond 500 epochs on A100.

11 test samples cover: short idiomatic phrases, multi-clause sentences, conversational passages, short narratives (Ramfal), and two ~5-min long-form stories (Surta/deer, Raghubeeer/Munna dialogue) — stable synthesis throughout.

Deployment — FastAPI Web Service on GCP

Research model (LLaMA QLoRA) → Production model (Gemma-4 GGUF): 4-bit CPU inference

Frontend	Backend	Translation	TTS	Hosting
HTML/CSS (index.html)	FastAPI + Uvicorn	Gemma-4 E4B GGUF Q4	Coqui VITS checkpoint	Modal
Text input, audio playback, dark/light mode	Port 8080, async handlers, CORS open	llama-cpp-python, CPU float32, ~4s/req	best_model_16731.pth, 22,050 Hz WAV	L4 GPU

API Endpoints

GET	/health	Model readiness probe
POST	/api/translate	Hindi → Haryanvi text
POST	/api/tts	Haryanvi → WAV download
POST	/api/pipeline	Full pipeline → audio URL
POST	/api/pipeline/base64	Full pipeline → base64 JSON

System Architecture — End-to-End Component Map

From user input to synthesised Haryanvi audio — every component and handoff

User Browser
HTML UI

FastAPI Backend
Port 8080

Gemma-4 GGUF
Translator

Haryanvi Text
Devanagari

VITS TTS Model
22,050 Hz

Model Weights
`model_weights/llm + tts`

Config Module
`app/config.py`

Tmp Audio Dir
`/tmp/audio_outputs`

Health Endpoint
`GET /health → models_ready`

Background loading

Models load on daemon thread
Server accepts connections immediately

Async dispatch

run_in_executor keeps event loop unblocked during inference

Devanagari validation

Regex Unicode property check
Retry with temp bump if fails

Audio lifecycle

UUID-keyed temp files
Cleaned on server shutdown

Error handling: HTTP 503 while loading • 400 for empty input • 404 for missing audio • 500 on model failure • /health returns {status, models_ready, error}

Live Demo — Deployed System Outputs

Five representative inputs and their live Gemma-4 GGUF Haryanvi outputs

Hindi Input	Haryanvi Output (Deployed — Gemma-4 GGUF)
तुम क्या कर रहे हो?	तू के कर रया सै?
मुझे पानी चाहिए	म्हने पाणी चाहिये
वह कहाँ गया	ओ कड़े ग्या
मुझे समझ नहीं आया	म्हने समझ कोन्या आया
तुम स्कूल क्यों नहीं गए?	तू स्कूल क्यूँ कोन्या गया?

Example API Request (curl)

```
code

curl -X POST \
  http://34.123.144.143:8080/api/pipeline \
  -H 'Content-Type: application/json' \
  -d '{"text":"तुम कहाँ जा रहे हो?"}'
```

JSON Response

```
code

{
  "hindi":     "तुम कहाँ जा रहे हो?",
  "haryanvi":  "तू कड़े जा रया सै?",
 }"audio_url": "/api/audio/<uuid>"
```

Code & Reproducibility — Environment Setup

Step-by-step: clone, install, configure, and launch

code

```
# 1. Clone the repository
git clone <repo-url> hindi-haryanvi-tts-vm
cd hindi-haryanvi-tts-vm
# 2. Create Python 3.12 virtualenv
python3.12 -m venv venv && source venv/bin/activate
```

code

```
# 3. Install pinned dependencies
pip install -r requirements.txt

# 4. Verify torch + CUDA
python -c "import torch; print(torch.cuda.is_available())"
```

Required Environment Variables

code

```
export LLM_BASE_MODEL_ID=./model_weights/llm/base
export TTS_CHECKPOINT=./model_weights/tts/best_model_16731.pth
export TTS_CONFIG=./model_weights/tts/config.json
export DEVICE=cpu    # cpu or cuda
export HF_HUB_OFFLINE=1  # no network calls at runtime
```

Launch

code

```
uvicorn app.main:app --host 0.0.0.0 --port 8080 --workers
1
```

code

```
screen -S server  # persist after SSH disconnect
```

Code & Reproducibility — Configuration Module

app/config.py — all paths and hyperparameters centralised; override via environment variables

```code
class ModelConfig:
  # LLM (deployed inference)
  LLM_BASE_MODEL_ID = os.getenv(
    'LLM_BASE_MODEL_ID',
    'meta-llama/Llama-3.1-8B-Instruct'
  )
  LLM_TEMPERATURE = float(
    os.getenv('LLM_TEMPERATURE', '0.1')  # deterministic
  )
  LLM_MAX_NEW_TOKENS = int(
    os.getenv('LLM_MAX_NEW_TOKENS', '256')
  )
  # TTS checkpoint
  TTS_CHECKPOINT = Path(os.getenv(
    'TTS_CHECKPOINT',
    './model_weights/tts/best_model_16731.pth'
  )
  # Server
  HOST = os.getenv('HOST', '0.0.0.0')
```

Reproducibility Controls

- Fixed random seed (42) for all train/val/test splits
- requirements.txt fully pinned — Python 3.12, all versions locked
- LLM temperature 0.1 — deterministic inference every run
- Model checkpoints on HuggingFace Hub (versioned, hashed)
- Per-experiment artefacts: hyperparameters.json, run_manifest.json
- Training notebooks in notebooks/ — fully self-contained
- Docker support — Cloud Run migration ready

Model weights directory structure: model_weights/llm/base/gemma-4-E4B-it-Q4_K_S.gguf • model_weights/tts/best_model_16731.pth • model_weights/tts/config.json

Code & Reproducibility — Inference Pipeline Code

Full pipeline endpoint — app/main.py (abridged)

code

```
@app.post('/api/pipeline')
async def pipeline(req: TextIn):
    _require_models()           # 503 until ready
    if not req.text.strip():
        raise HTTPException(400, 'Empty input')

    loop = asyncio.get_running_loop()

    # Step 1: Hindi → Haryanvi
    haryanvi = await loop.run_in_executor(
        None, translate,
        req.text, _llm, _tokenizer
    )

    # Step 2: Haryanvi → Audio
    audio_id = str(uuid.uuid4())
    path = TMP_AUDIO_DIR / f'{audio_id}.wav'
    await loop.run_in_executor(
        None, synthesize, _tts, haryanvi, str(path)
```

Translator (Gemma GGUF)

code

```
# app/models/translator.py
def translate(text, llm, tok):
  prompt = build_prompt(text)
  out = llm(prompt,
    temperature=0.1,
    top_p=0.95,
    repeat_penalty=1.05
  )
  return validate_devanagari(out)
```

Devanagari Validation

code

```
# Regex Unicode property check
# If ratio < 0.5 → inject failure
# msg and retry with temp=0.3
# to break hallucination loop
```

Code & Reproducibility — Training Notebooks

Two self-contained Jupyter notebooks for full pipeline reproduction

notebooks/llama-text-pipe-final.ipynb

1. Load Satyam-Srivastava/RDS dataset from HuggingFace Hub
2. Merge Hindi/Haryanvi columns, filter, deduplicate, length-check
3. Apply LLaMA 3.1 chat template + assistant-only loss masking
4. Run 6 QLoRA experiments (baseline → higher_rank)
5. Save per-experiment: hyperparameters.json, validation_predictions.csv, qualitative_predictions.csv
6. Upload LoRA adapter to HuggingFace Hub

notebooks/coqui-tts-pipe-final.ipynb

1. Load ankitdhiman/haryanvi-tts from HuggingFace
2. 6-step preprocessing: transliteration → NFC → filter → resample → trim → LJSpeech export
3. Load SYSPIN/vits_Hindi_Female — verify weight transfer per component
4. Configure VITS trainer (GAN config, LRs, FP16, batch=16)
5. Run 300-epoch training on T4 GPU — checkpoint every 1,680 steps
6. Export best_model_16731.pth + config.json to HuggingFace

All experiment artefacts saved to Google Drive: combined_outputs/llm/ and tts_output/vits_haryanvi_ai4bharat/ — linked in Appendix

Project Artifacts — All Deliverable Links

HuggingFace Hub, Google Drive, and live deployment endpoints

HuggingFace Dataset

Parallel Text Dataset

`huggingface.co/datasets/Satyam-Srivastava/RDS`

HuggingFace Model

VITS TTS Checkpoint

`Satyam-Srivastava/TTS_haryanvi_bangru_vits_based_model`

HuggingFace Model

LLaMA LoRA Adapter

`Satyam-Srivastava/hindi-to-haryanvi-translation-llama3-qlora`

HuggingFace Space

MUSHRA Evaluation Space

`Satyam-Srivastava/mushra-haryanvi-tts-evaluation`

HuggingFace Space

TTS Demo Space

`Satyam-Srivastava/tts-haryanvi-bangru-text-to-audio`

Live Demo

Live FastAPI Server

`https://abhishekbidhan11--api.modal.run/`

Google Drive

LLM Training Outputs

`Google Drive: combined_outputs/llm/`

Google Drive

TTS Training Outputs

`Google Drive: tts_output/vits_haryanvi_ai4bharat/`

Google Drive

Generated Audio Samples

Future Work — Roadmap by Expected Impact

Six concrete directions from demo-quality to production-ready

#1 **Extend TTS Training (500+ Epochs, A100)** — HIGH

Mel loss projected to reach 8–10 range — clear jump from intelligible to fully natural phrasing. ~5 hrs on A100 vs ~42 hrs on T4.

#2 **Expand Corpus via Back-Translation** — HIGH

Augment 5,594 pairs with back-translated data — directly fixes 3 systematic failure modes: idioms, proper nouns, complex sentences.

#3 **Fix Evaluation Pipeline (LoRA Loader)** — HIGH

Resolve LoRA adapter caching bug — unblocks valid BLEU/chrF comparison across experiments and enables proper model selection.

#4 **Formal MOS, WER, MCD Evaluation** — MEDIUM

Native Haryanvi listener panel + automated metrics via IndicWav2Vec ASR and librosa MCD. Replaces internal self-evaluation with reproducible benchmarks.

#5 **GPU-Enabled Cloud Run Deployment** — MEDIUM

Sub-second inference latency. Docker image already built — Cloud Run migration prepared. Enables concurrent users at scale.

#6 **Multi-Speaker TTS + Real-Time Streaming** — LOW

Expand from SYSPIN-derived single voice to multi-speaker. Add streaming inference for truly real-time educational content delivery.

Future Work — Extended Research Directions

Longer-term opportunities beyond the immediate roadmap

Other Haryanvi Sub-Dialects

Bangru is one of several Haryanvi varieties. Extending to Ahirwati, Bagri, and Mewati dialects would cover the full Haryana linguistic landscape. Same pipeline architecture applies — only the parallel corpus changes.

Educational Content Platform

Partner with NGOs and state education departments to deliver dialect-authentic audio content to rural schools. The pipeline is already API-first — integration is primarily a content and distribution problem.

Crowdsourced Haryanvi Corpus

Deploy a data collection web app to Haryanvi-speaking communities. Even 10 hours of native speaker audio would dramatically close the Hindi lexical bias gap and unlock multi-speaker training.

Larger Base LLM for Translation

Gemma-4 9B or LLaMA 3.3 70B GGUF models would provide richer linguistic representations. Prompt-engineering the dialect rules into a larger base gives headroom without the compute cost of full fine-tuning.

Cross-Lingual Transfer

The QLoRA + VITS approach is language-agnostic. The same methodology applies to Rajasthani, Bhojpuri, Maithili, and other low-resource Indo-Aryan varieties — a scalable template for dialect NLP research.

Real-Time Streaming TTS

Integrate streaming VITS inference to deliver partial audio as it is generated. Combined with GPU-accelerated Cloud Run, this enables truly interactive conversation-speed dialect synthesis — <200ms TTFA.

Limitations — Known Constraints & Mitigations

Limited Training Data

~5,594 text pairs and ~1.5 hrs of audio. Well below the 10+ hr minimum for scratch VITS training. Transfer learning mitigates this but Hindi bias remains.

Fix: Back-translation augmentation + crowdsourced corpus collection planned for v2.

Hindi Lexical Bias in TTS

Decoder occasionally reverts to Standard Hindi phonology on formal or rare tokens due to dominant Hindi acoustic priors inherited from the SYSPIN donor model.

Fix: Extended training beyond 500 epochs + larger Haryanvi corpus. Bias reduces as Haryanvi patterns accumulate.

Evaluation Pipeline Bug

LoRA adapter-loading failure in evaluation script caused BLEU/Exact-Match to be identical across all experiments. Eval loss values remain valid.

Fix: PeftModel inference script rewrite is the top-priority fix in the next sprint.

CPU Inference Latency ~4s/req

Gemma-4 GGUF on CPU takes ~4 seconds per translation request — acceptable for demo, insufficient for production scale or real-time use.

Fix: GPU-enabled Cloud Run deployment already Docker-prepared — sub-second with A100.

Single Speaker Voice

VITS model produces only the SYSPIN Hindi Female voice profile adapted to Haryanvi. No speaker variety, no gender variation.

Fix: Multi-speaker VITS training with a crowdsourced multi-speaker Haryanvi corpus.

No Formal Native Listener Study

MUSHRA evaluation used a structured framework but not a formal panel of native Haryanvi speakers — results are indicative, not statistically certified.

Fix: Formal MOS study with native listener panel planned as future evaluation milestone.

Conclusion — What We Achieved

Translation

chrF 77.65, BLEU 71.76 — captures है→सै, नहीं→कोनी, रहा→रया and 8 other systematic Haryanvi substitutions across diverse sentence types

Speech

Intelligible, naturally-prosodied Haryanvi speech after 300 epochs. Stable across 11 samples including two ~5-minute stories. MUSHRA MOS 86.0.

Evaluation

MUSHRA MOS 86.0 — near-natural quality confirmed. 42.5-point gap over degraded anchor. Low σ=3.0 confirms consistency. chrF/BLEU for translation.

Deployment

Live FastAPI on GCP. Gemma-4 GGUF for CPU inference. Five REST endpoints. Docker-ready. Reproducible with pinned requirements and fixed seeds.

Key Innovations

- Rule-constrained synthetic data encoding 11 Haryanvi grammar categories into prompts — first such dataset
- Character-level TTS tokenisation — bypasses espeak-ng's dialect mispronunciations entirely
- CPU-efficient GGUF quantisation for accessible live deployment without GPU infrastructure
- Production-ready FastAPI web service with async inference — not just a research notebook

A small, focused team can build a genuinely dialect-authentic speech system for an underserved language in a few months, using open-source models, modest compute, and disciplined engineering.

Live Project Links

All four deployments are publicly accessible

TTS Deployment — **Haryanvi Text-to-Speech**

Enter Haryanvi text and hear the synthesised dialect-authentic speech output.

`huggingface.co/spaces/Satyam-Srivastava/tts-haryanvi-bangru-text-to-audio`

MUSHRA Evaluation — **Human Evaluation Interface**

Listen to Reference, System, and Anchor audio samples and rate speech quality 0–100.

`huggingface.co/spaces/Satyam-Srivastava/mushra-haryanvi-tts-evaluation`

Full Pipeline — GCP — **End-to-End Live Demo**

Enter Hindi text → get Haryanvi translation + synthesised WAV audio. Full FastAPI pipeline on GCP.

`https://abhishekbidhan11--api.modal.run/`

HuggingFace Dataset — **Hindi–Haryanvi Parallel Corpus**

5,594 parallel pairs used to train the translation model. Publicly available for research.

`huggingface.co/datasets/Satyam-Srivastava/RDS/tree/main`

Thank You

Regional Dialect Synthesis Pipeline • Hindi → Haryanvi → Speech

Abhishek
22f3000978

Satyam Srivastava
21f1000629

Sanket Agrawal
23f1001709

Md Fazlur Rahman
23f1001897

77.65	71.76	86.0	300	5,594
chrF	BLEU	MUSHRA MOS	Epochs	Train Pairs

Group 12 • IIT Madras DS & AI Lab • April 2026

A study in low-resource dialect engineering

References

Works cited across all six milestones

[1] AI4Bharat (2023). Indic-TTS [Computer software]. GitHub: https://github.com/AI4Bharat/Indic-TTS

[2] Casanova, E. et al. (2024). XTTS: A massively multilingual zero-shot TTS. arXiv:2406.04904

[3] Dettmers, T. et al. (2023). QLoRA: Efficient finetuning of quantized LLMs. arXiv:2305.14314

[4] Grattafiori, A. et al. (2024). The Llama 3 herd of models. arXiv:2407.21783

[5] Kim, J., Kong, J., & Son, J. (2021). VITS: Conditional VAE with adversarial learning for TTS. arXiv:2106.06103

[6] Mehta, S. et al. (2023). Matcha-TTS: Fast TTS with conditional flow matching. arXiv:2309.03199

[7] Ren, Y. et al. (2020). FastSpeech 2: Fast and high-quality end-to-end TTS. arXiv:2006.04558

[8] Vaswani, A. et al. (2017). Attention is all you need. NeurIPS Vol. 30

[9] Javed, T. et al. (2022). Towards building ASR systems for the next billion users. AAAI, 36(10)

[10] Mohanta, A. et al. (2026). Towards prosodically informed Mizo TTS. arXiv:2601.02073

[11] Bhashini (2024). FastSpeech2 model for Indian languages. IndiaAI AIKosh

All project code, notebooks, weights, and documentation are publicly available — see Project Artifacts slide for HuggingFace and Google Drive links.